Fundamental Sorting Techniques

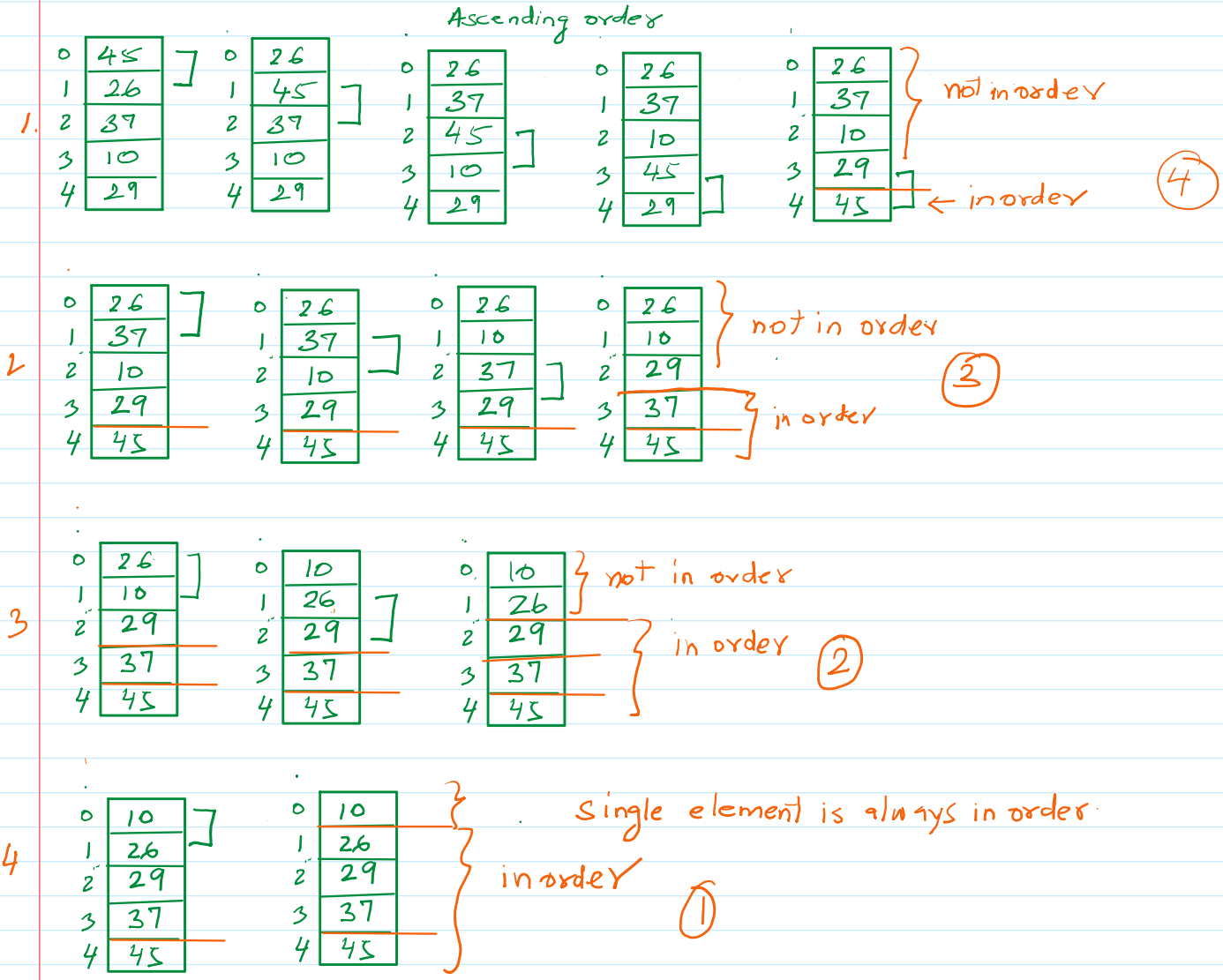
14 March 2026 20:18

Sorting means arrangement of data either in ascending order or in descending order.
Fundamental Sorting algorithms are:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort

These sorting techniques is applied only for small amount of data.

Bubble Sort:
Compares adjacent elements and swap them if they are in wrong order.

Ascending order



Single element is always in order.

how many comparisons are needed to sort the array?

For 5 elements we need = 4 + 3 + 2 + 1 = 10 comparisons.

For 6 elements we need = 5 + 4 + 3 + 2 + 1 = 15 comparisons

For 7 elements we need = 6 + 5 + 4 + 3 + 2 + 1 = 21 "

For 7 elements we need $= 6+5+4+3+2+1 = 21$

For n " " " $= (n-1) + \dots + 6 + 5 + 4 + 3 + 2 + 1 =$

Sum of natural numbers.

$$C_N = \frac{N \times (N+1)}{2} = \frac{(n-1)(n-\cancel{1}+\cancel{1})}{2}$$

$$\boxed{\text{No. of comparisons in bubble sort} = \frac{n(n-1)}{2}}$$

highest degree term.

Big-O → $f(x) = x^2 + 2x + 1$ $O(x^2)$

In Big-O notation we select the highest degree term from the expression. And, we discard the constants also.

notation represents worst case scenario.

$$\frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{n}{2} = O(n^2)$$

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 10
void printArray(int arr[], char msg[]){
    int i;
    printf("\n%s\n[", msg);
    for(i = 0; i < SIZE - 1; i++){
        printf("%d, ", arr[i]);
    }
    printf("%d]\n", arr[i]); //last element
}
void bubbleSort(int A[]){
    int i, j, t;
    for ( i = 0; i < SIZE - 1 ; i++)
    {
        for ( j = 0; j < SIZE - 1 - i; j++)
        {
            if(A[j] > A[j+1])
            {
                t = A[j];
                A[j] = A[j+1];
                A[j+1] = t;
            }
        }
```

Home work

Q What logic we write such that our algorithm will realize that all the elements are in order, and it stops execution. [We cannot compromise more time by adding another loop.]

```
    }

}
int main(int argc, char const *argv[])
{
    int A[SIZE], i;
    srand(time(NULL)); // Seed with current time
    for(i = 0; i < SIZE; i++){
        A[i] = rand() %101 ;
    }
    printArray(A, "Before sorting A[]:");
    bubbleSort(A);
    printArray(A, "After sorting A[]:");
    return 0;
}
```

Output:

```
Before sorting A[]:
[73, 50, 61, 30, 51, 78, 100, 3, 52, 52]

After sorting A[]:
[3, 30, 50, 51, 52, 52, 61, 73, 78, 100]
```